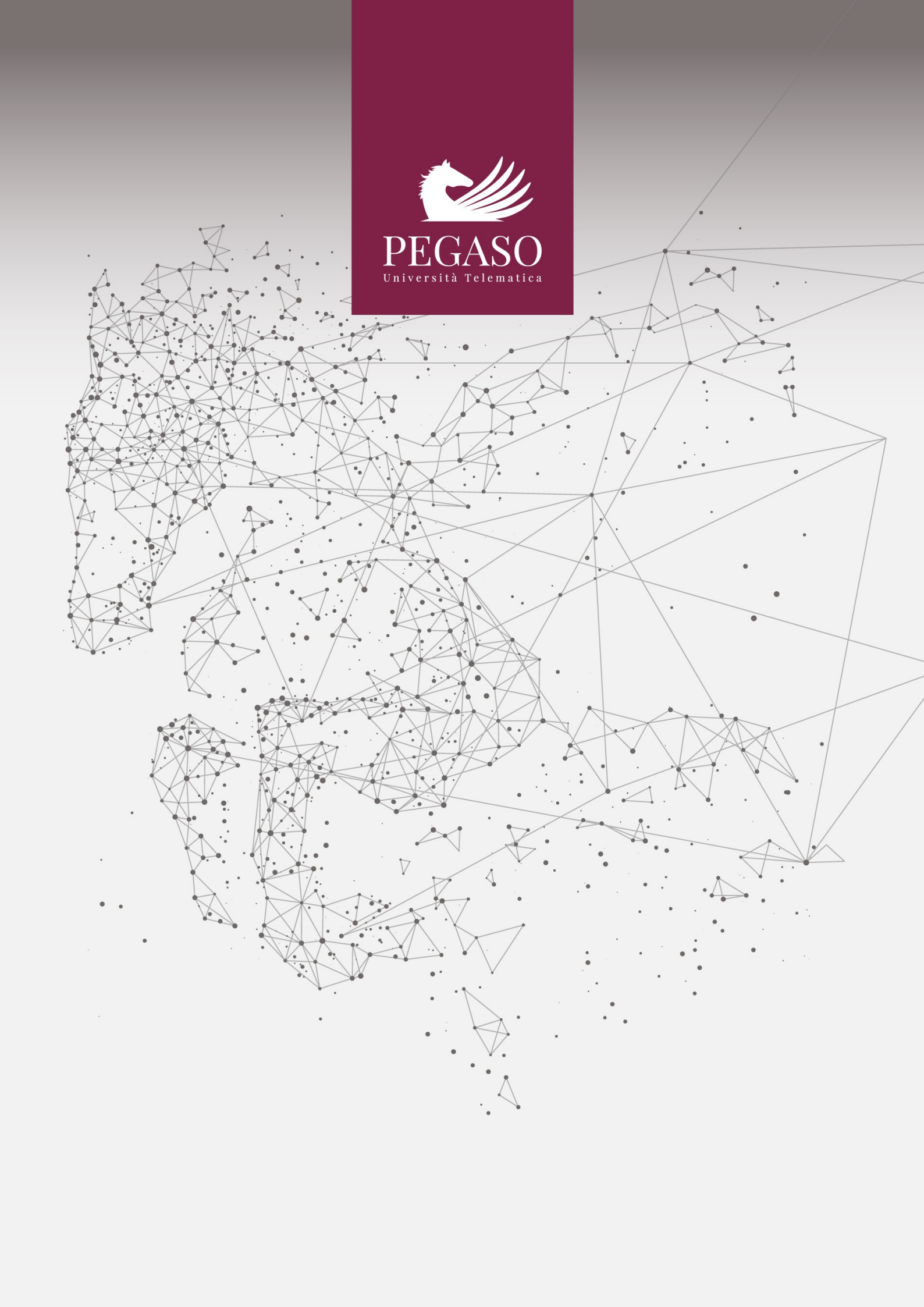

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. INPUT, ELABORAZIONE E OUTPUT 4
3. VARIABILI E TIPI DI DATO 6
4. INPUT DA TASTIERA 9
BIBLIOGRAFIA 11

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si illustra la struttura di un tipico programma suddiviso in raccolta dell'input, elaborazione e visualizzazione dell'output. Si introducono, inoltre, le stringhe e le loro peculiarità in Python, i commenti per la "descrizione" del codice e si approfondisce la funzione print per la visualizzazione dell'output a schermo. Infine, si forniscono le regole per la denominazione delle variabili in Python, si presentano i principali tipi di dato delle variabili e si forniscono le indicazioni per poter acquisire dati in input da tastiera.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Input, elaborazione e output

Generalmente, un programma è suddiviso in tre fasi ben distinte:

1. riceve input (da tastiera, da file, ecc.);
2. esegue un'elaborazione dell'input ricevuto (es. calcoli matematici);
3. produce output (solitamente visualizzato a schermo).

Volendo fornire un esempio molto semplice come il calcolo dell'area di un rettangolo:

- la base e l'altezza del rettangolo costituiranno l'input;
- il programma elaborerà questi dati moltiplicandoli tra loro;
- il risultato del calcolo verrà presentato a schermo.

In Figura 1, si riporta un semplice schema relativo alle fasi tipiche di un programma e all'esempio appena descritto.

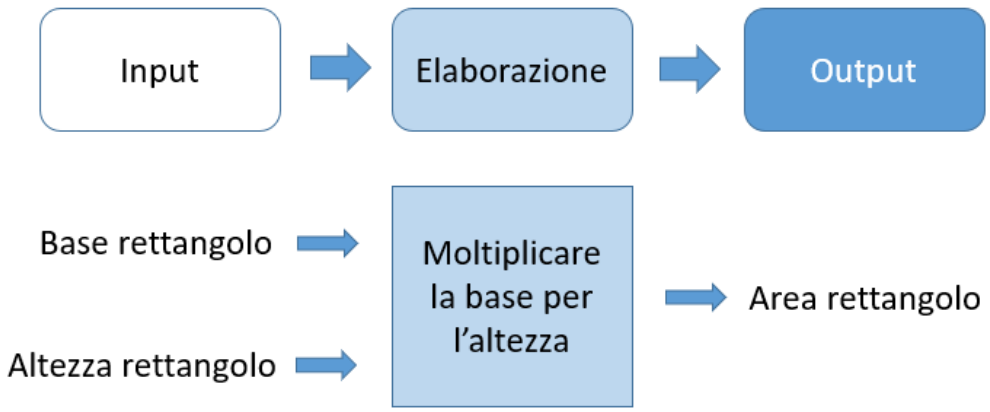


Figura 1. Schema delle fasi di esecuzione di un programma

2.1 Introduzione alle stringhe

I programmi lavorano sempre con dati di qualche tipo: uno dei tipi più utilizzati è rappresentato dalle **stringhe** (o **sequenze di caratteri**). Le stringhe devono essere racchiuse tra **virgolette** per contrassegnare l'inizio e la fine della stringa stessa. "*Ciao mondo!*" e "*Via Roma, 1*" sono, infatti due esempi di stringhe.

L'utilizzo delle **virgolette doppie** al posto dell'**apostrofo** è necessario per permettere proprio l'utilizzo di apostrofi nella stringa che altrimenti verrebbero identificati come terminatori di stringa.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Se ci fosse la necessità di inserire in una stringa le virgolette doppie (es. per una citazione) per contrassegnare l'inizio e la fine di una stringa si utilizzano le triple virgolette """.

""" *Sul cancello c'era scritto "cave canem"* """ è un esempio di stringa con citazione e uso di apostrofo. Le triple virgolette possono, inoltre, racchiudere stringhe su più righe come ad esempio:

"""Procedi
sulla stessa
strada"""

Per visualizzare l'output a schermo è possibile utilizzare la funzione print. In particolare, tra parentesi andrà inserita la stringa da stampare utilizzando le regole per le stringhe appena analizzate. Un esempio potrebbe essere la seguente istruzione:

```
print(""" Sul cancello c'era scritto "cave canem" """ )
```

che, come risultato a schermo, produrrebbe:

```
 Sul cancello c'era scritto "cave canem"
```

2.2 I commenti

A volte potrebbe nascere l'esigenza di commentare il codice spiegando lo scopo di una o più istruzioni all'interno dello stesso codice sorgente. A tale scopo vengono utilizzati i **commenti,** delle brevi note inserite nel codice sorgente che vengono ignorate dall'interprete Python. I commenti, contrassegnati dal carattere #, vengono utilizzati dal programmatore e non dall'utente del programma. Ciascun commento, che verrà visualizzato in rosso nell'IDLE, può essere inserito in qualsiasi punto del codice ed inizia la sua funzione dal carattere successivo al carattere speciale # e termina con la riga in cui è stato inserito.

Solitamente i commenti vengono inseriti o all'inizio del codice sorgente per spiegare l'utilità del programma, oppure dopo le istruzioni per spiegare l'utilità della specifica istruzione. Questi ultimi prendo il nome di **end-line comment**. Il seguente esempio mostra l'utilizzo di entrambi i tipi di commenti:

```
# Questo programma visualizza le informazioni di una automobile

print("Utilitaria")  # visualizza la tipologia di automobile
print("Benzina")     # visualizza il tipo di carburante
```

Anche se l'utilità dei commenti nel codice sorgente può sembrare superflua, l'uso appropriato dei commenti risulta essere fondamentale nel caso in cui il programma sia caratterizzato da una maggiore complessità o nel caso in cui il codice sorgente debba essere modificato a distanza di tempo dalla sua scrittura.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Variabili e tipi di dato

Una **variabile** è un nome che rappresenta un valore nella memoria di un computer. L'operazione fondamentale che riguarda le variabili è l'**assegnazione** il cui formato generale è:

nome_della_variabile = espressione

Un esempio di assegnazione può essere:

iva = 22

in cui la variabile *iva* assume il valore alla destra del simbolo di uguaglianza.

Al fine di utilizzare correttamente una variabile, è necessario ricordare che:

- una variabile può essere utilizzata esclusivamente se le è stato assegnato un nome e un valore;
- non deve essere riportata tra le virgolette come avviene per le stringhe;
- può essere passata come argomento ad una funzione tra parentesi.

In Python, come per altri linguaggi di programmazione, esistono delle regole da seguire per denominare le variabili. Innanzitutto, le keyword di Python (le più usate sono riportate in Figura 2) non possono essere utilizzate come nomi di variabili.

Il primo carattere del nome di una variabile deve necessariamente essere una lettera o un underscore _ (trattino basso), mentre per i caratteri successivi è possibile usare lettere, numeri e underscore.

Inoltre, i nomi delle variabili non possono contenere spazi e sono **case sensitive**: la variabile alfa è, infatti, diversa dalla variabile Alfa.

and	continue	else	for
import	not	raise	assert
def	except	from	in
or	return	break	del
exec	global	is	pass
try	class	elif	finally
if	lambda	print	while

Figura 2. Principali keyword di Python

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ci sono, poi, delle ulteriori indicazioni per la denominazione delle variabili in Python. Il nome di una variabile deve essere significativo e far capire il suo uso nel programma. Infine, per quanto riguarda i nomi per variabili complesse o composte, è convenzione per i programmatori Python non utilizzare nomi CamelCase (es. retribuzioneLorda, retribuzioneOraria), ma trattini bassi tra le parole che compongono il nome della variabile (es. retribuzione_lorda, retribuzione_oraria).

3.1 L'output di più variabili e stringhe

Python permette di visualizzare più elementi con una singola chiamata alla funzione **print**.

Gli elementi da visualizzare vengono separati da virgole quando vengono passati come argomento. Gli argomenti verranno visualizzati nell'ordine in cui vengono passati alla funzione. La funzione print, di default, inserisce uno spazio nella visualizzazione a schermo tra gli elementi passati come argomento. Nell'esempio successivo si vuole stampare a schermo una stringa fissa e una parte che dipende dal contenuto di una variabile (nell'es. area):

```
area = 15
print("L'area del rettangolo è pari a ",area)
```

che, come risultato a schermo, produce:

```
L'area del rettangolo è pari a  15
```

3.2 Tipi di dato

Le variabili possono assumere diversi valori mentre il programma viene eseguito. Risulta essere fondamentale, quindi, conoscere quali tipi di dato sono previsti dal linguaggio di programmazione Python. In particolare, per i dati numerici il tipo **int** rappresenta il tipo di dato da utilizzare per numeri interi, mentre **float** il tipo da utilizzare per i numeri in virgola mobile.

Si fa presente che non è possibile scrivere simboli di valuta, spazi o virgole nei **letterali numerici**, cioè nei numeri scritti direttamente nel codice sorgente (es. prezzo = € 5,45 è errato).

Il tipo di dato, al contrario di altri linguaggi di programmazione come C++ e Java, non viene esplicitato in Python.

Per conoscere il tipo di una variabile esiste la funzione **type**() che restituisce una stringa in cui è contenuto il tipo della variabile stessa. Per poter visualizzare a schermo il tipo di una variabile, la funzione type diventerà un parametro della funzione print (composizione di funzioni). Nell'esempio che segue, alla

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

variabile area viene assegnato il valore numerico intero 15 e subito dopo viene visualizzata a schermo la stringa restituita dalla funzione type applicata sulla variabile area.

```
area = 15
print(type(area))
```

Come è possibile intuire, il risultato a schermo sarà:

```
<class 'int'>
```

Generalmente, una variabile può riferirsi ad un elemento di qualsiasi tipo. Infatti, ad una variabile a cui è stato assegnato un tipo, può esserne assegnato un altro. Nell'esempio che segue alla variabile area viene assegnato prima il valore intero 15 (si chiede di visualizzare il tipo della variabile area) e, successivamente, il valore in virgola mobile 15.0 (si chiede, nuovamente di visualizzare il tipo della variabile area).

```
area = 15
print(type(area))
area = 15.0
print(type(area))
```

Il risultato delle due chiamate alla funzione print sarà:

```
<class 'int'>
<class 'float'>
```

Oltre ai tipi numerici, Python dispone del tipo **str** per le stringhe. Nell'esempio che segue viene assegnato il valore "Claudio" alla variabile nome, viene stampato a schermo il tipo della variabile nome (trattandosi di una stringa a schermo verrà visualizzato 'str'). Successivamente, viene assegnato il valore "Loconsole" alla variabile cognome. Infine, viene richiamata la funzione print passandogli 3 parametri: la stringa "Mi chiamo", la variabile nome e la variabile cognome.

```
nome = "Claudio"
print(type(nome))
cognome = "Loconsole"
print("Mi chiamo", nome, cognome)
```

Il risultato sarà il seguente:

```
<class 'str'>
Mi chiamo Claudio Loconsole
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Input da tastiera

La maggior parte dei programmi necessita della lettura di dati di input forniti dall'utente. A tale scopo, Python offre la funzione built-in **input**() che legge l'input da tastiera e ritorna un dato di tipo stringa contenente tutto ciò che l'utente ha digitato prima di premere il tasto Invio. Il formato di un'istruzione per l'input da tastiera è il seguente:

nome_variable = input(prompt)

dove il **prompt** è tipicamente una stringa che spiega all'utente che dato deve inserire. Si evidenzia che, a differenza della funzione print che inserisce spazi tra gli argomenti prima della visualizzazione a schermo, la funzione input di default non mostra uno spazio dopo la stringa prompt. Un esempio di programma con input da tastiera e output è il seguente: alla variabile *nome* viene assegnata la stringa di ritorno della funzione input che, a sua volta, prima della digitazione dell'utente mostra a schermo come prompt la domanda "Come ti chiami?". Il programma procede con la stampa a schermo della stringa "Ciao" seguita dal contenuto della variabile nome. Il programma termina visualizzando il tipo della variabile nome (str). Il codice sorgente Python corrispondente è:

```
nome = input("Come ti chiami? ")
print("Ciao", nome)
print(type(nome))
```

che mostra come risultato:

```
Come ti chiami? Claudio
Ciao Claudio
<class 'str'>
```

La funzione input, infatti, restituisce sempre una **stringa** (tipo di dato str).

Nei casi in cui fosse necessario dover inserire dei valori numerici da parte dell'utente del programma e non delle stringhe, è fondamentale l'utilizzo combinato delle funzioni built-in che permettono di convertire i tipi di dato. In gergo informatico il cambio di tipo di dato di una variabile viene chiamato **casting**.

Nel dettaglio, la funzione **int(nome_var)** permette di convertire nome_var (es. una stringa) in un intero (int), mentre la funzione **float(nome_var)** permette di convertire nome_var (es. una stringa) in un float (float).

Sfruttando la funzione input() per l'inserimento di dati da parte dell'utente in maniera combinata con una delle due funzioni di casting (int o float), sarà possibile, quindi, inserire come input dei valori numerici. L'uso combinato delle funzioni, più propriamente definito come **chiamata di funzioni nidificate (nested function call)** ha il seguente formato generico:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

funzione1(funzione2(argomenti))

il valore che ritorna la funzione2 viene passato alla funzione1. Nel nostro caso, l'utilizzo della funzione input come funzione2 e della funzione int o float come funzione1, ci permette di prendere la stringa restituita dalla funzione input e convertirla in numero.

La conversione di tipo di dato, però, viene effettuata solo se il numero digitato (parametro passato alla funzione int o float) corrisponde effettivamente ad un numero, altrimenti viene lanciata un'eccezione (errore a run-time del programma).

Un esempio dell'utilizzo di funzioni nidificate può essere il seguente: la variabile importo nel primo caso è una stringa (come visibile dalla prima stampa del tipo di variabile), mentre nel secondo caso, la variabile importo è effettivamente un intero. La differenza principale tra i due tipi di variabile è che solo con variabili intere o in virgola mobile è possibile eseguire calcoli (es. somma). Il codice sorgente dell'esempio è:

```
#in questo caso l'importo è una stringa
importo = input("Quanto vuoi prelevare? ")
print("Hai prelevato", importo, "€")
print(type(importo))

print()

#in questo caso l'importo è un valore
importo = int(input("Quanto vuoi prelevare? "))
print("Hai prelevato", importo, "€")
print(type(importo))
```

Il risultato del programma è:

```
Quanto vuoi prelevare? 10
Hai prelevato 10 €
<class 'str'>

Quanto vuoi prelevare? 10
Hai prelevato 10 €
<class 'int'>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).